Tunable bank + off-mesh quadrature

Progress toward the JCP paper

16 October 2026

Numbers from the lanes' generated reports. Slides 1–6 and 8 use the earlier first-order references (provisional); slide 7 uses the new order-verified second-order references.

The idea: evaluate the nonlinear term off the mesh

- **Before:** the reduced model's nonlinear integral $\int \psi\, u(u_x+u_y)$ was computed *on the simulation mesh* (dense sum, fitted EQ, or a giant precomputed tensor). Cost and error both depended on the mesh.
- **Hari's idea:** our bank $u(x) = G(x)c$ is a continuous function. Evaluate it and its *exact* autodiff derivative at a few thousand textbook quadrature points (Gauss, lattice):

$$N(c) = P^{\top}\left[(Bc) \odot (Dc)\right]$$

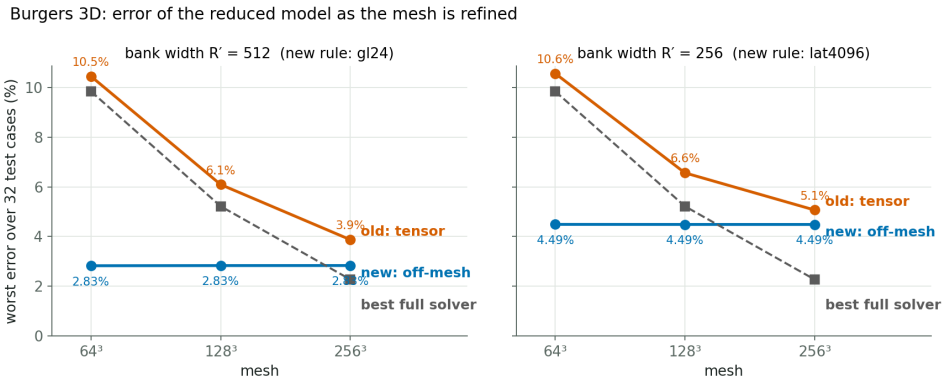
  with B, D, P precomputed once offline.
- No fitting, no per-mesh retraining, and the same points at every mesh.

What we did since bringing in Hari's quadrature

1. **Plugged Hari's off-mesh rules into our tunable model** for Burgers 2D and 3D, and ran every rule family as a full PDE solve: Gauss, lattices (Fibonacci, CBC, Korobov), Sobol, Smolyak, and the flux form, against the old mesh methods (dense sum, mesh lattice, exact tensor).
2. **Found why it works** (mechanism study): exact derivative vs. mesh differencing.
3. **Fixed the next bottleneck:** second-order time stepping (CN-R, BDF2, Galerkin).
4. **Tested what else limits accuracy:** a wider bank (up to 1024 shapes) and smoother or derivative-trained banks.
5. **Built trustworthy references:** order-verified second-order solutions, cross-checked by a spectral solver; sealed held-out cohorts.
6. **Combined the winners** (second-order stepping with the wide bank) and scored them against the new references.
7. **Now running:** 3D speed (one-pass solver, narrower bank, batching, large-mesh comparison against spectral).

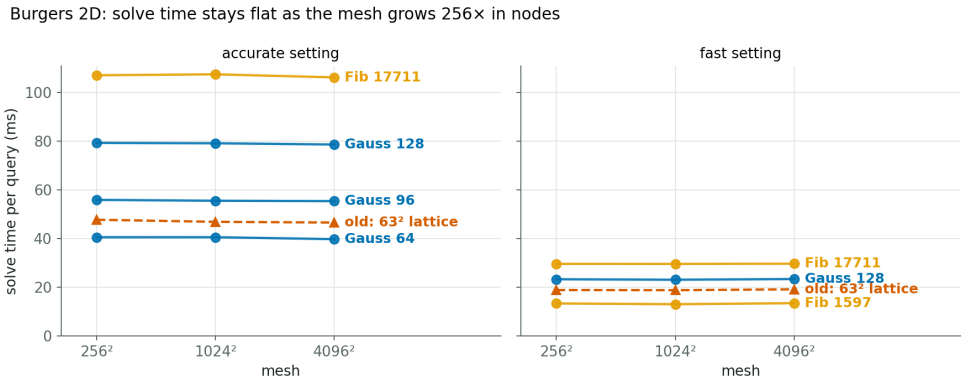
Every step was pre-registered and adversarially checked by an independent reviewer (Codex) before it ran.

1. Accuracy no longer depends on the mesh (3D Burgers)



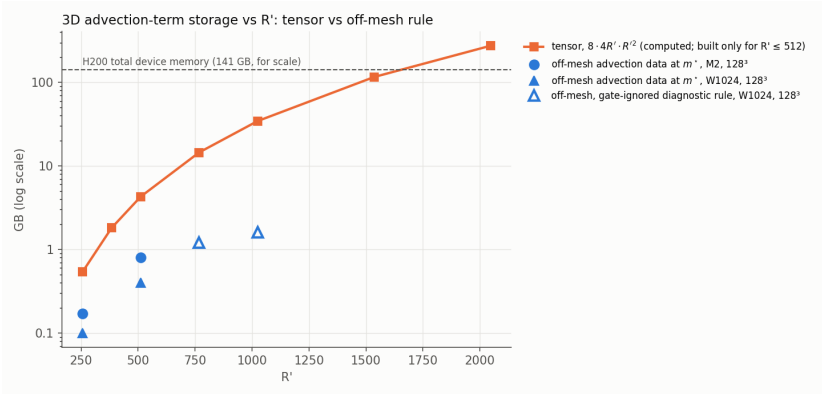
Off-mesh: **2.83 % at every mesh**. The old tensor goes 10.5 → 6.1 → 3.9 %.

2. Neither does cost (2D Burgers)



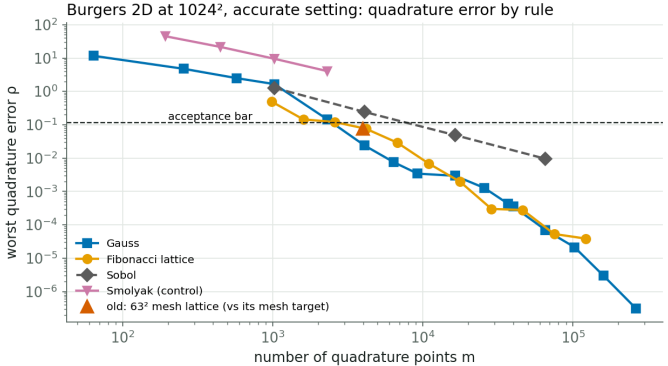
Solve time is identical from 256^2 to 4096^2, a mesh $256\times$ larger.

3. Memory: wide banks become possible



The tensor needs 34 GB at width 1024 and exceeds a whole H200 past about 1600. Off-mesh stays under 2 GB.

4. Which quadrature rules work



Gauss and lattices converge fast. Sobol needs about 4× more points; Smolyak fails. Deployed: Gauss (accurate setting), lattice (fast setting).

Every rule as a full Burgers solve: accuracy vs. speed

3D, held-out test cases (64), bank 512 — worst / typical error is the same at every mesh for off-mesh rules; speed-up vs. the cheapest equally accurate full simulation (same job, same GPU)

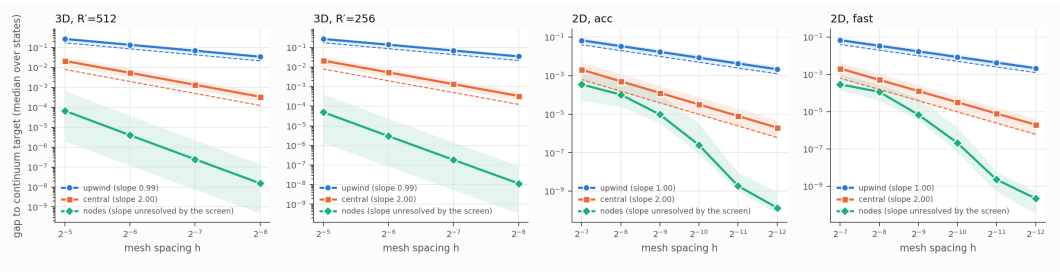
Rule	points	worst / typical	64^3	128^3	256^3
Old exact tensor	–	3.9 / 1.3 % (at 256^3)	0.16×	0.53×	2.2×
Lattice	4,096	**2.8 / 1.1 %**	0.18×	0.54×	**3.1**×
Gauss 24^3 (deployed)	13,824	2.8 / 1.1 %	0.11×	0.34×	2.6×
Gauss 32^3	32,768	2.8 / 1.1 %	0.06×	0.20×	1.9×
Sobol	16,384	3.3 / 1.2 %	0.10×	0.32×	2.5×
Smolyak / lattice 256	small	**fail** (37–42 %)			

2D, held-out test cases (64), accurate setting — worst error at 256^2/1024^2/4096^2; speed-up at 4096^2

Rule	points	worst error	speed-up 4096^2
Full simulation	–	5.5 / 2.9 / 2.4 %	1×
Old mesh lattice 63^2	3,969	5.6 / 3.3 / 1.6 %	11×
Gauss 64^2	4,096	**2.6 / 2.6 / 1.6 %**	**13**×
Fibonacci	6,765	2.6 / 2.6 / 1.6 %	10×
Sobol	4,096	2.8 / 2.8 / 1.6 %	–
Gauss 32^2	1,024	**fail** (9.0 / 8.8 / 2.7 %)	–

Every working rule gives the same accuracy; speed depends on mesh size. We are slower on small meshes and faster on large ones. All numbers use first-order references (provisional).

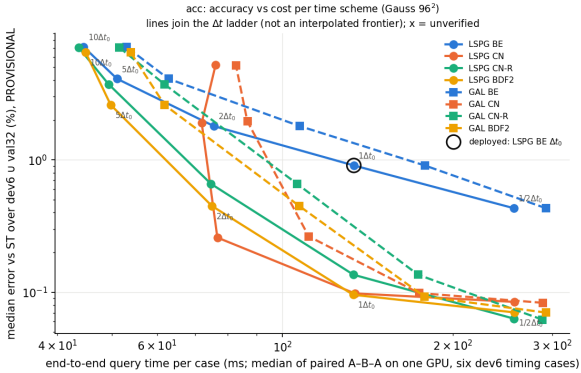
5. Why it works: the exact derivative



Bands: 10th–90th percentile over the fixed states. Dashed: C-pos manufactured state (upwind, central). Slopes fitted on the pre-registered window only (the coarsest mesh is excluded); the nodes curve is not a fit.

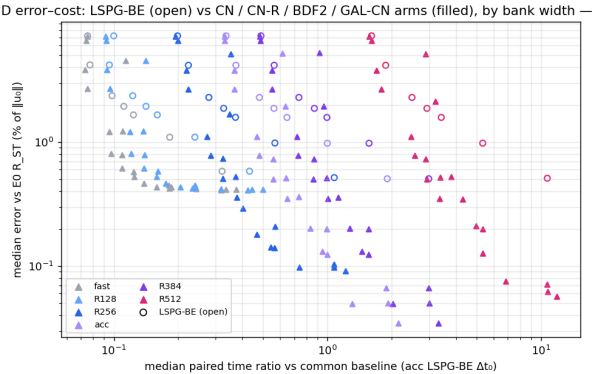
The old error was the mesh's own differencing error: slope 1 for upwind, 2 for central, exactly as theory says. Mesh nodes with the exact derivative reproduce the off-mesh result, so the win is the exact derivative, and quadrature makes it cheap (about 14k points against millions of nodes).

6. Next bottleneck, time stepping: fixed with second-order schemes



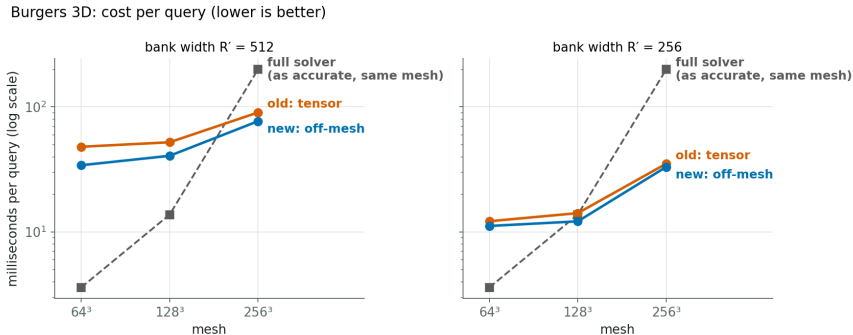
At equal cost, second-order schemes (CN-R, BDF2) have about 9× less error than backward Euler (circled).

7. Where we are: against new order-verified references



2D: median error 0.98 → 0.20 % at equal cost; about **14**× faster than a full simulation that is second order in time. 3D: median 1.5 → 0.12 % at equal cost.

8. Honest limit: 3D speed



We only win at large meshes ($2.6\times$ at 256^3). Early, unaudited runs show a spectral solver beating us clearly at 64^3. Large-mesh comparisons (128^3–384^3) are running now.

Next steps

- 3D speed lane (running): one-pass solver at big steps, narrower bank, batching, large-mesh comparison against spectral.
- Standard ROM baselines: POD with DEIM, ECSW and GNAT.
- A harder example where spectral solvers struggle: non-box geometry, sharp fronts, non-polynomial nonlinearity.
- Open the sealed test cohorts for the final paper numbers.

Done this cycle: order-verified second-order references (3D clean pass; 2D pass, 13 of 204 cases flagged), and second-order stepping combined with the wide bank.